

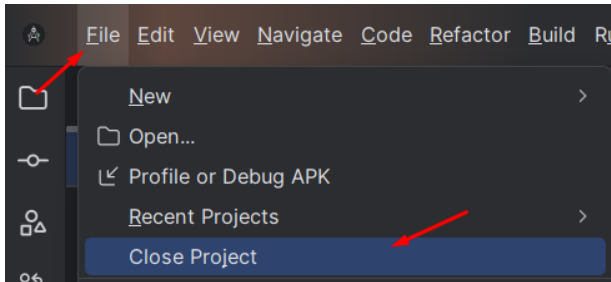
Лабораторная работа №2. Создание приглашения для героя

PixLetter: Часть I

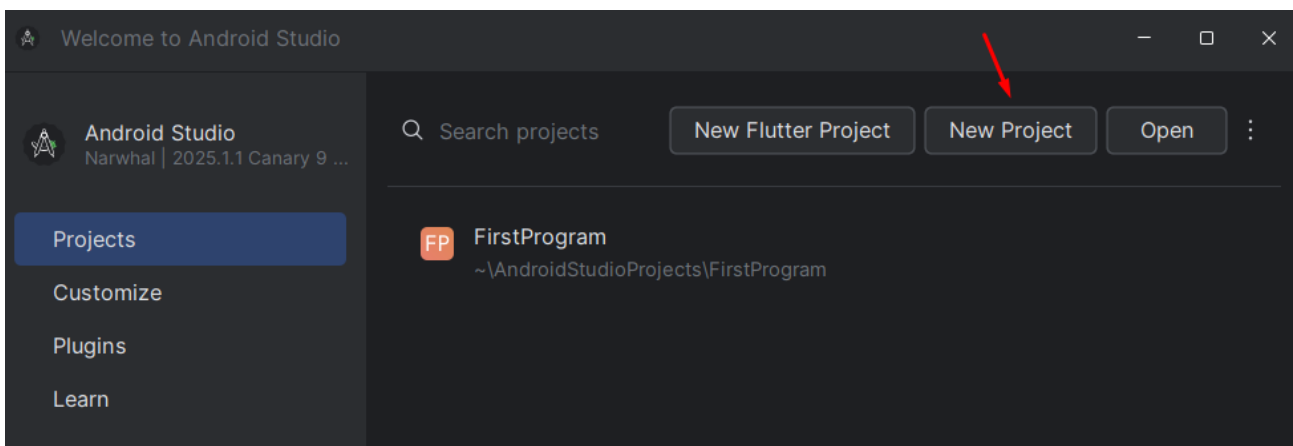
Шаг 1. Создаём проект

1. Запустите Android Studio.

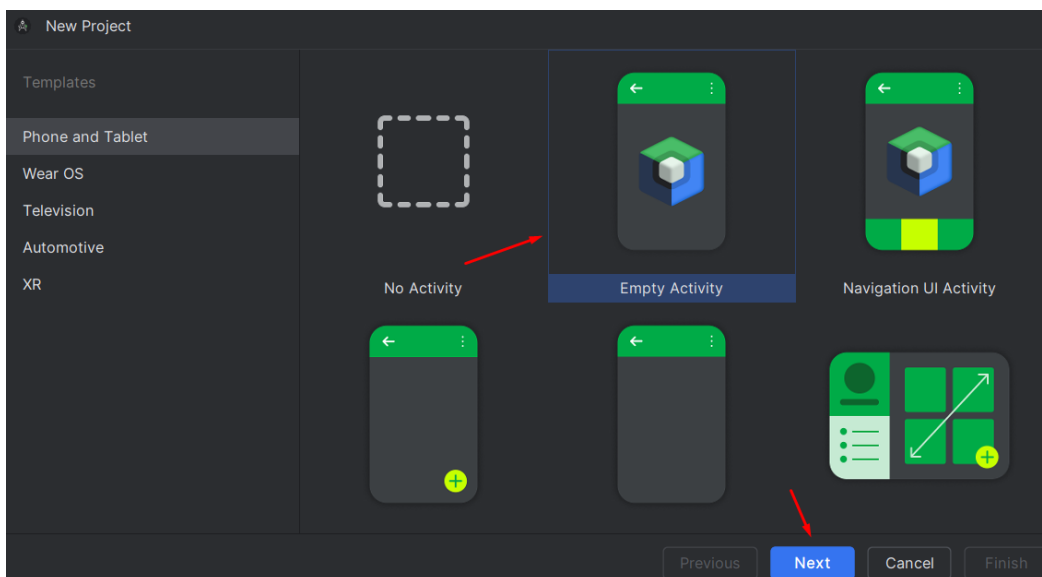
Если у вас автоматически открылся предыдущий проект, необходимо закрыть его, чтобы перейти в стартовое окно. Для этого нажмите **File** → **Close Project**. Также можно сразу создать новый проект через **File** → **New** → **New Project**.



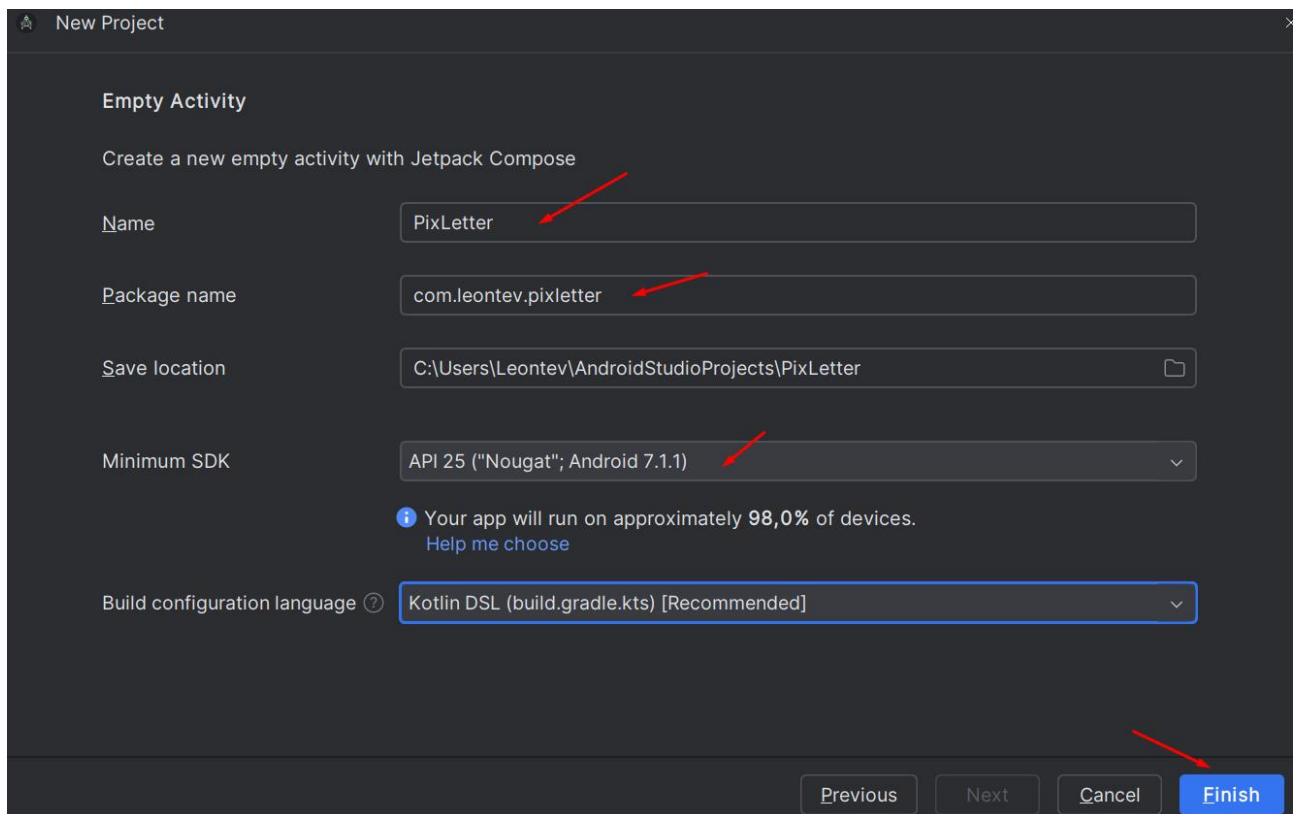
2. В окне "Welcome to Android Studio" нажмите New Project.



3. Выберите шаблон Empty Activity и нажмите Next.



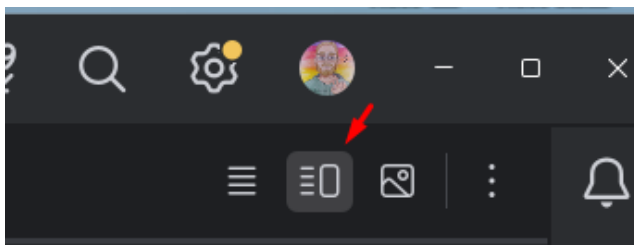
4. В появившемся окне настройки проекта заполните поля следующим образом:



- **Name:** введите **PixLetter**.
- **Package name:** замените второе слово на свою фамилию (например, com.ivanov.pixletter).
- **Save location:** оставьте без изменений, но запомните путь — там будут храниться все файлы проекта.
- **Language:** выберите **Kotlin** (если поле отображается).
- **Minimum SDK:** выберите **API 25: Android 7.1.1 (Nougat)**.

Нажмите **Finish** и дождитесь завершения сборки проекта.

5. Переключитесь на режим Split, нажав кнопку **Split** в правом верхнем углу **Android Studio** — это позволит одновременно видеть **код и дизайн**.



Шаг 2. Разбираем код

1. Откройте файл MainActivity.kt — он откроется автоматически. Обратите внимание на автоматически сгенерированные функции - **onCreate()** и **setContent()**:

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PixLetterTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(paddingValues = innerPadding)
                    )
                }
            }
        }
    }
}

```

- **onCreate()** — точка входа в Android-приложение. Здесь инициализируются компоненты интерфейса. В отличие от обычных Kotlin-программ, где используется `main()`, в Android используется именно `onCreate()`.

- **setContent()** — вызывается внутри `onCreate()` и используется для описания интерфейса с помощью **Composable-функций**.

2. Функция Greeting()

Найдите в коде функцию **Greeting()**. Она имеет аннотацию **@Composable**, что означает, что её можно использовать для генерации интерфейса с помощью **Jetpack Compose**:

```

2 Usages
33 @Composable
34 fun Greeting(name: String, modifier: Modifier = Modifier) {
35     Text(
36         text = "Hello $name!",
37         modifier = modifier
38     )
39 }

```

Особенности **@Composable-функций**:

```

2 Usages
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(

```

1. Перед объявлением обязательно указывается аннотация **@Composable**.
2. Названия принято начинать с заглавной буквы.
3. Не возвращают значения напрямую, а описывают, **что должно отображаться на экране**.

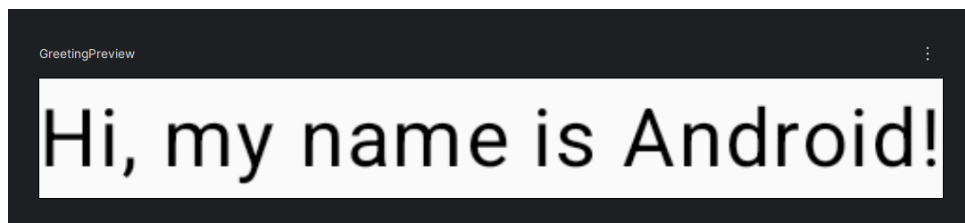
1. Функция должна быть предварена аннотацией **@Composable**.
2. Имена **Composable-функций** принято писать с заглавной буквы.
3. **@Composable-функции** не возвращают значений

Сейчас функция **Greeting()** принимает имя и отображает приветствие этому человеку.

Обновите Greeting(), чтобы вместо "Hello" отображалось ваше имя и фраза приветствия:

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hi, my name is $name!",
        modifier = modifier
    )
}
```

Android Studio должен автоматически обновить предварительный просмотр:



3. Функция GreetingPreview()

Эта функция предназначена для предварительного просмотра интерфейса без запуска всего приложения. Она также имеет аннотацию **@Preview**.

Аннотация **@Preview** может принимать параметры, например:

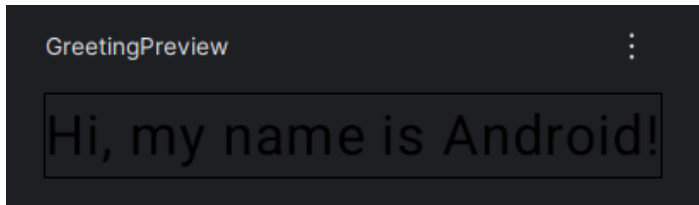
```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    PixLetterTheme {
        Greeting(name = "Android")
    }
}
```

Как вы можете видеть, **@Preview** аннотация принимает параметр с именем **showBackground**. Если **showBackground** установлено значение **true**, то будет добавлен фон к вашему компоуемому предварительному просмотру.

Измените значение showBackground на false, чтобы посмотреть, как изменится внешний вид в предварительном просмотре.

```
@Preview(showBackground = false)
```

Если у вас включена тёмная тема, то вы заметите разницу:



Затем верните значение **showBackground** обратно на **true**.

Добавьте своё имя в вызов **Greeting()** внутри **GreetingPreview()**:

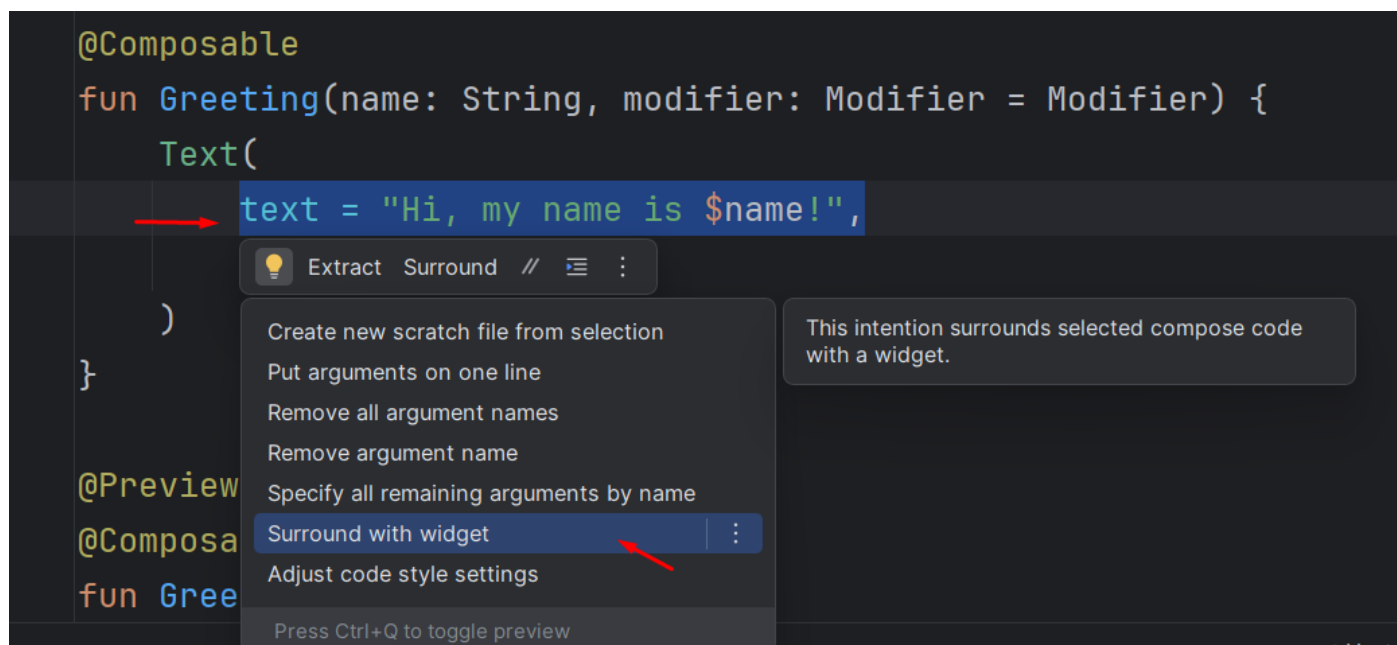
```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    PixLetterTheme {
        Greeting(name = "Denis")
    }
}
```

Шаг 3. Меняем фон через Surface

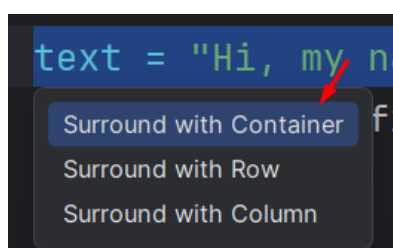
Теперь мы хотим изменить фон компонента. Для этого нужно обернуть **Text()** в **Surface** — это контейнер, внешний вид которого можно настраивать (цвет, форма и т.д.).

1. Оберните Text() в контейнер:

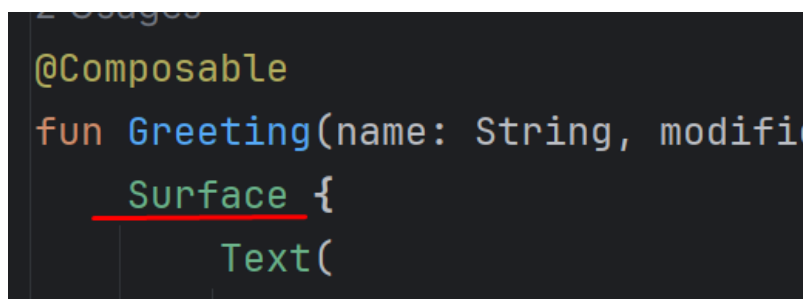
- Выделите строку **Text(...)**.
- Нажмите **Alt+Enter** → выберите **Surround with widget**.



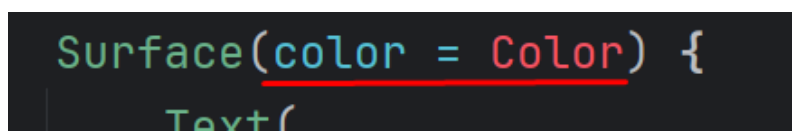
- Выберите **Surround with Container**:



2. По умолчанию добавится **Box**, но вы можете заменить его на **Surface()**:



3. Добавьте в контейнер параметр **color** и задайте ему значение **Color**:



Когда вы напишите **Color**, то можете заметить, что слово подсвечено красным. Это значит, что **Android Studio** не видит нужный импорт. Для решения этой проблемы существует два способа. Первый способ предполагает, что вы знаете какой библиотеки вам не хватает. Прокрутите до верхней части файла, где у нас идут различные **import**.

Добавьте следующую строку в блок **import** (вверху файла):

import androidx.compose.ui.graphics.Color

```

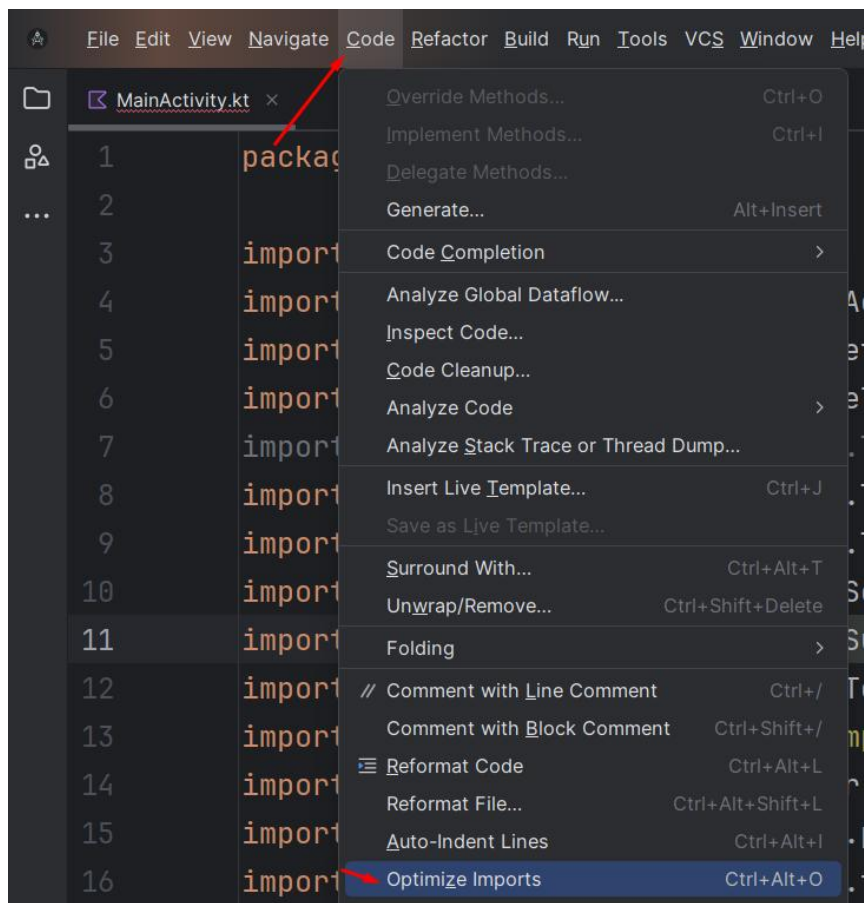
1 package com.example.firstprogram
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.Box
8 import androidx.compose.foundation.layout.fillMaxSize
9 import androidx.compose.foundation.layout.padding
10 import androidx.compose.material3.Scaffold
11 import androidx.compose.material3.Surface
12 import androidx.compose.material3.Text
13 import androidx.compose.runtime.Composable
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.tooling.preview.Preview
16 import com.example.firstprogram.ui.theme.FirstProgramTheme
17 import androidx.compose.ui.graphics.Color
18

```

В коде рекомендуется **хранить импорты в алфавитном порядке** и **удалять неиспользуемые**, чтобы проект оставался чистым и читаемым.

Для этого:

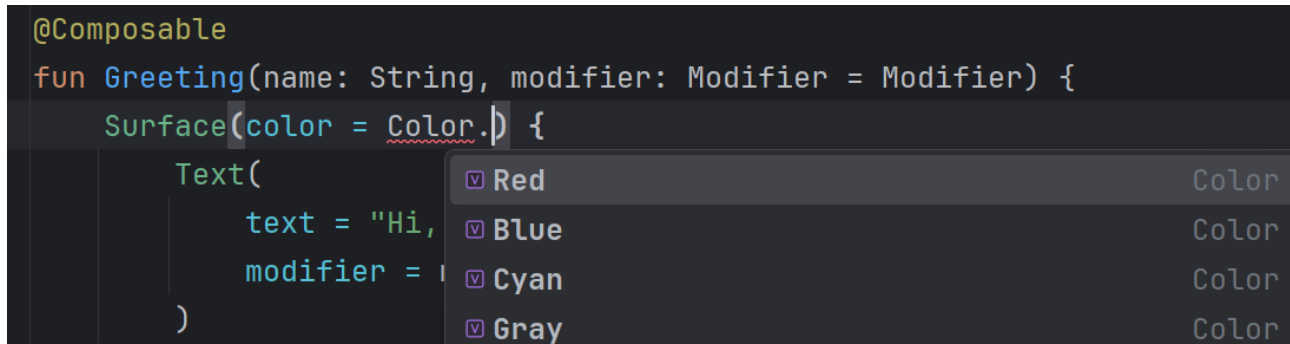
- Перейдите в верхнее меню **Code** → **Optimize Imports** или используйте сочетание клавиш **Ctrl+Alt+O**.



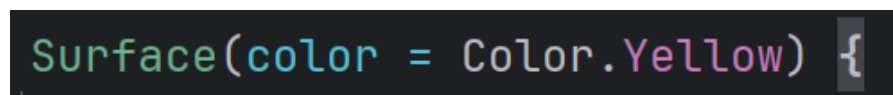
Если после ввода цвета в скобках `Surface(color = ...)` он остаётся **подчёркнутым красным**, это значит, что значение указано некорректно.

Для исправления:

- После `Color` поставьте точку (`.`), и появится всплывающее меню с выбором предопределённых цветов:



Выберите любой цвет, например жёлтый:

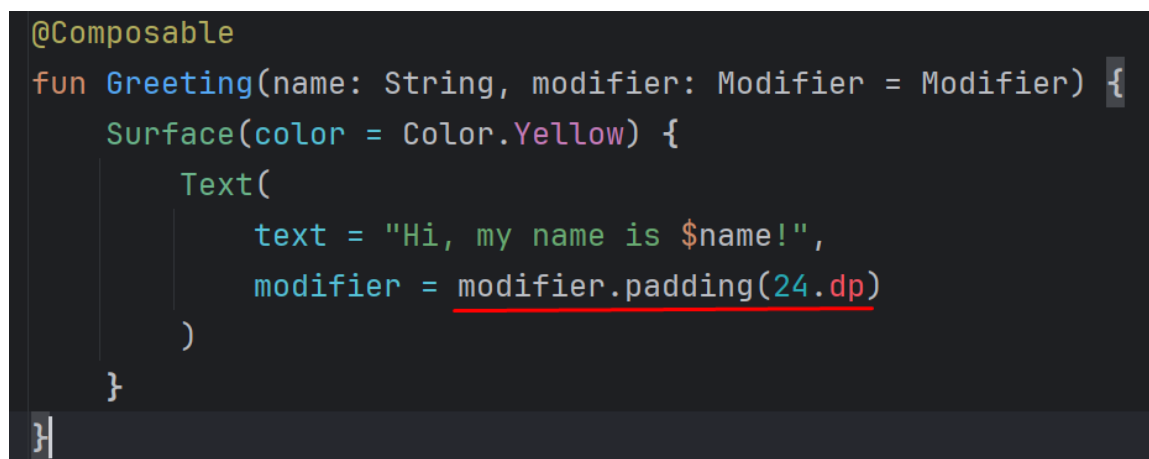


Шаг 4. Добавление отступов с помощью `Modifier.padding`

Чтобы добавить пространство вокруг текста, используйте модификатор `padding`.

Модификаторы в Jetpack Compose — это специальные параметры, применяемые к UI-элементам для настройки их внешнего вида и поведения.

- Добавьте модификатор к элементу `Text` с размером **24.dp**:



- Не забудьте импортировать единицу измерения:

`import androidx.compose.ui.unit.dp`

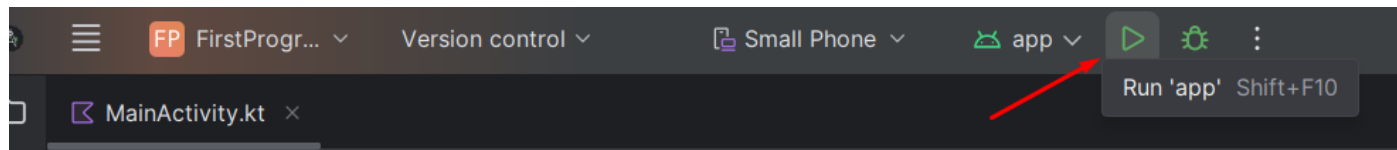
- После добавления нового импорта снова выполните **Optimize Imports** (`Ctrl+Alt+O`), чтобы отсортировать список по алфавиту.

Шаг 5. Запуск приложения

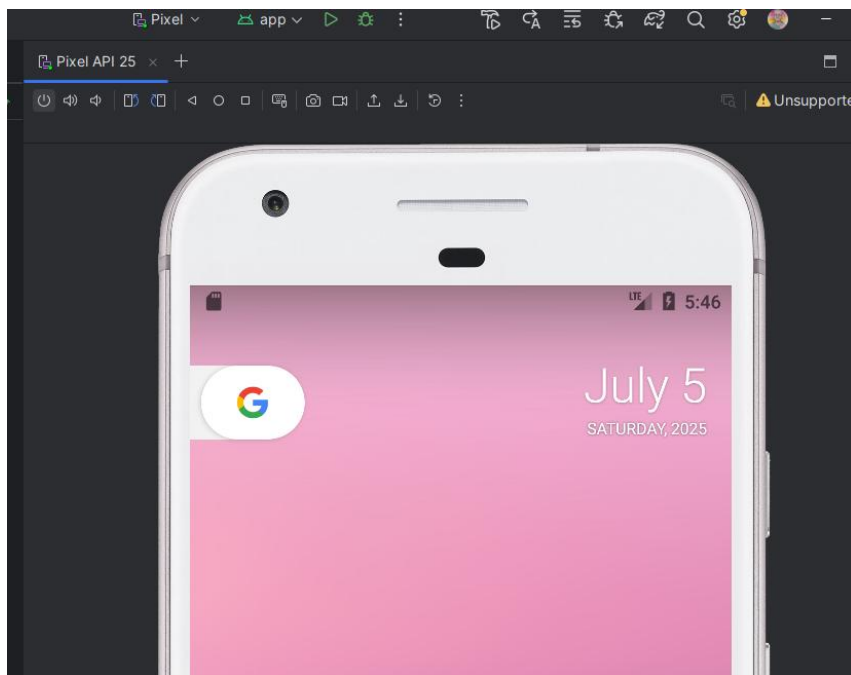
Теперь можно протестировать приложение на эмуляторе.

Если вы ещё **не настроили эмулятор**, обратитесь к отдельной инструкции по созданию виртуального устройства.

- Нажмите **зелёную кнопку запуска (Run)** в верхней панели.



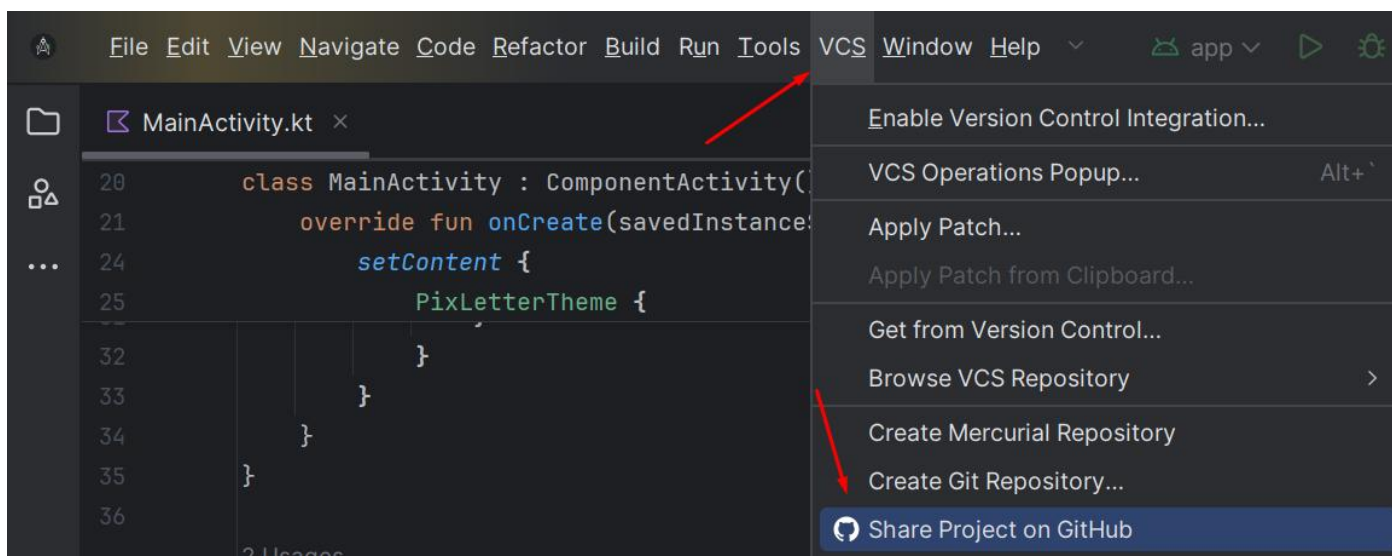
Первый запуск может занять несколько минут. После запуска **окно эмулятора** появится **рядом с Android Studio**. Оно работает так же, как и физическое устройство.



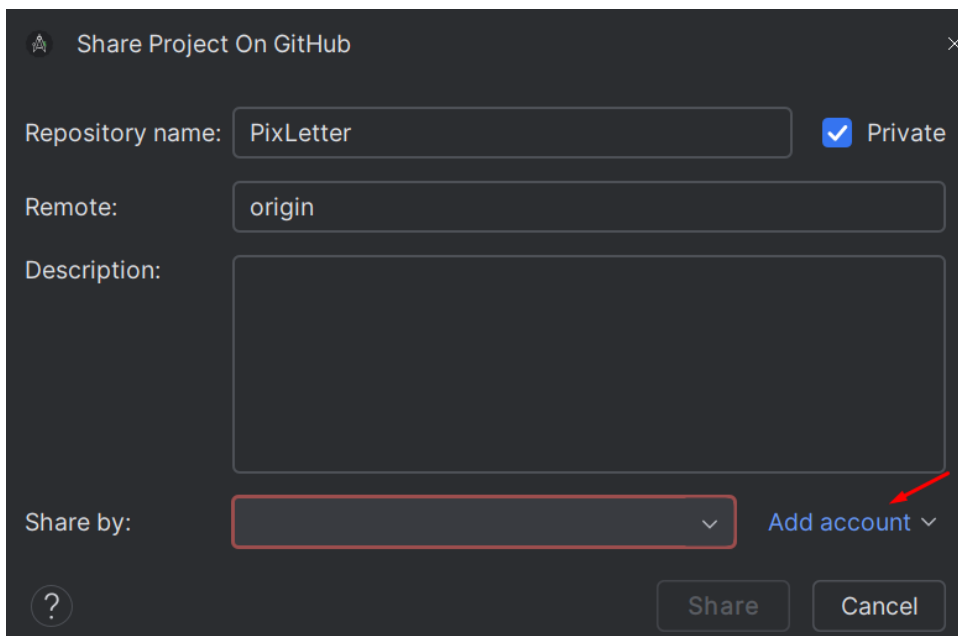
Шаг 6. Подключение проекта к GitHub

Теперь подключим ваш проект к **Git** и загрузим его на **GitHub**:

1. Перейдите в меню **VCS** → **Share Project on GitHub**.

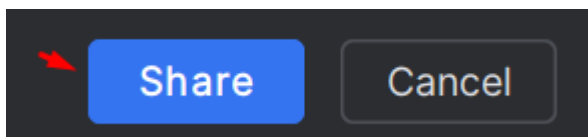


2. Если потребуется — выполните вход в GitHub-аккаунт:



3. Укажите имя репозитория (по умолчанию будет имя проекта).

4. Нажмите **Share**, чтобы отправить проект в облако:



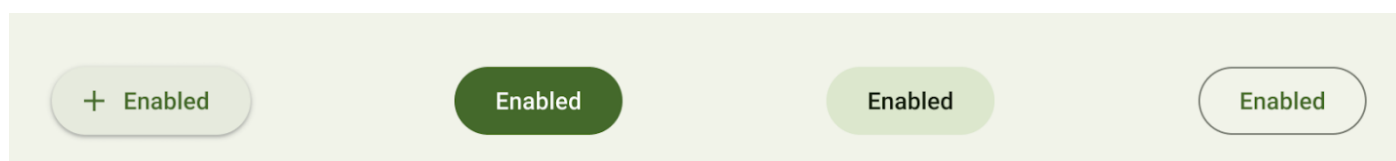
Шаг 7. Введение в Jetpack Compose

Что такое пользовательский интерфейс (UI)?

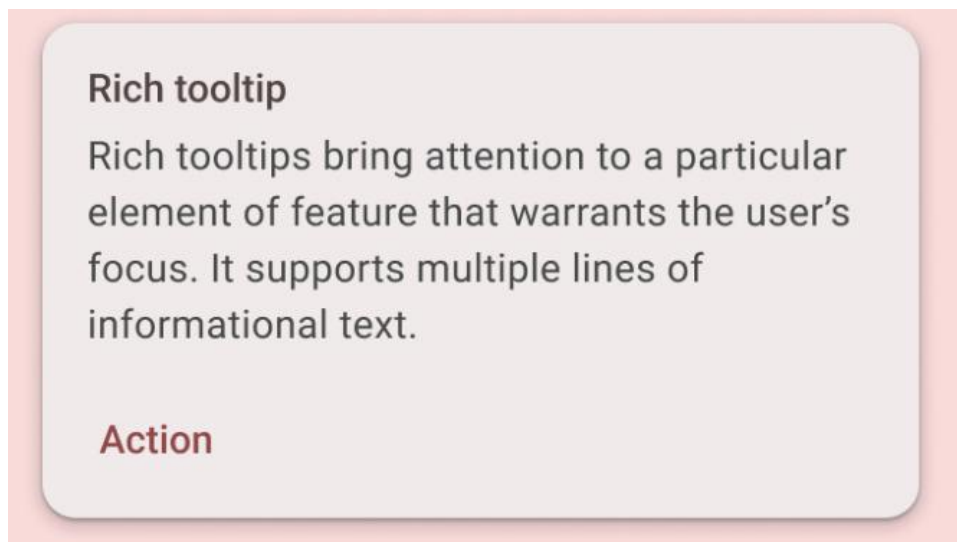
Пользовательский интерфейс (UI) — это всё, что отображается на экране приложения и с чем взаимодействует пользователь:

- **Кнопки**
- **Текст**
- **Изображения**
- **Поля ввода**
- и др.

На изображении содержится нажимаемая кнопка, текстовое сообщение и поле ввода текста, куда пользователи могут вводить данные.



Кнопка, на которую можно нажимать



Текстовое сообщение внутри блока



Поле ввода текста

Каждый из таких элементов называется **компонентом пользовательского интерфейса (UI-компонентом)**. Они могут быть:

- **Интерактивными** — кнопки, поля ввода;
- **Декоративными** — текст, изображения и т.д.

Что такое Jetpack Compose?

Jetpack Compose — это **современный фреймворк от Google для создания интерфейсов на Android**. Он использует **Kotlin** и предлагает:

- Меньше кода
- Удобный синтаксис
- Полную интеграцию с Android Studio
- Реактивный подход

Интерфейс в Compose строится с помощью **компокуемых функций** — они принимают входные данные и определяют, что должно отображаться на экране.

Компокуемые функции (@Composable)

Компокуемые функции — это строительные блоки интерфейса в Compose. Они:

- **Описывают UI**, не возвращая значения.

- Имеют аннотацию **@Composable**.
- Принимают параметры (например, `name: String`) и используют их для отображения.

Пример:

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")
}
```

Аннотации

Аннотации — это дополнительные инструкции для компилятора или других инструментов. Они начинаются с символа **@** и ставятся перед элементом кода (функцией, классом, свойством).

Следующая диаграмма представляет собой пример аннотированной функции:

Prefix character: @

Annotation

@Composable

fun Greeting(name: String, modifier: Modifier) {}

Function declaration

Аннотации с параметрами

Аннотации могут принимать параметры. Параметры предоставляют дополнительную информацию для инструментов, которые их обрабатывают. Ниже приведены некоторые примеры аннотаций **@Preview** с параметрами и без них.

```
44  @Preview
45  @Composable
46  fun GreetingPreview() {
47      HappyBirthdayTheme {
48          Greeting( name: "Android")
49      }
50  }
```

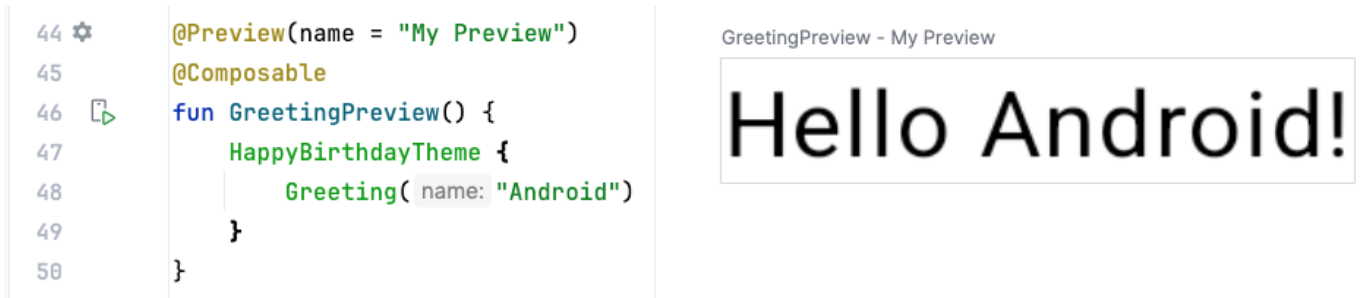
GreetingPreview

Hello Android!

Аннотация без параметров

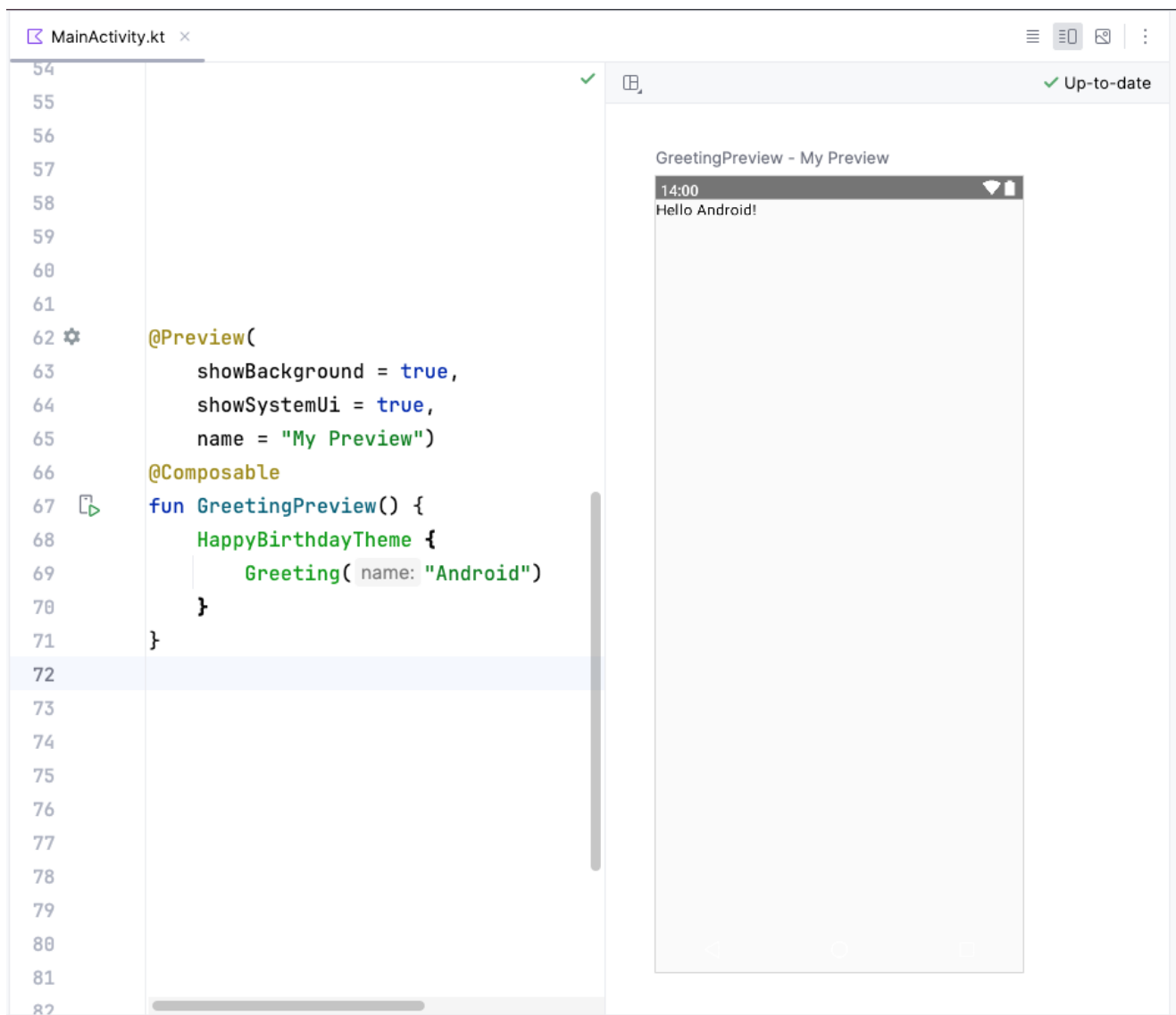


Фон предварительного просмотра аннотации



Аннотация с предварительным заголовком

Вы можете передать аннотации несколько аргументов:



Пример составной функции

Компонуемая функция аннотируется с помощью **@Composable**. Эта аннотация сообщает компилятору Jetpack Compose, что функция предназначена для **создания элементов пользовательского интерфейса (UI)**.

Переименование функции предварительного просмотра

Найдите функцию **GreetingPreview()**. Эта функция используется для предварительного просмотра UI, создаваемого функцией **Greeting()**.

Хорошей практикой является давать функциям **осмысленные имена**, отражающие их назначение. Переименуйте **GreetingPreview()** на **RoyalInvitationPreview()**:

```
@Preview(showBackground = true)
@Composable
fun RoyalInvitationPreview() {
    PixLetterTheme {
        Greeting(name = "Denis")
    }
}
```

Вложенные компонуемые функции

Компонуемые функции **могут вызывать другие** компонуемые функции. В данном случае функция предварительного просмотра вызывает функцию **Greeting()**.

По соглашению, **имена компонуемых функций** пишутся в **PascalCase**: каждое слово с заглавной буквы, без подчеркиваний.

Зачем не вызывать Greeting() напрямую?

Компонуемые функции должны **предоставлять значения по умолчанию для параметров**, если они вызываются в **@Preview**.

Поэтому **не рекомендуется напрямую вызывать Greeting() в @Preview**, если у неё есть параметры.

Вместо этого:

- Создайте отдельную функцию для превью — например, **RoyalInvitationPreview()**
- Внутри неё вызывайте **Greeting()** с заданным параметром.

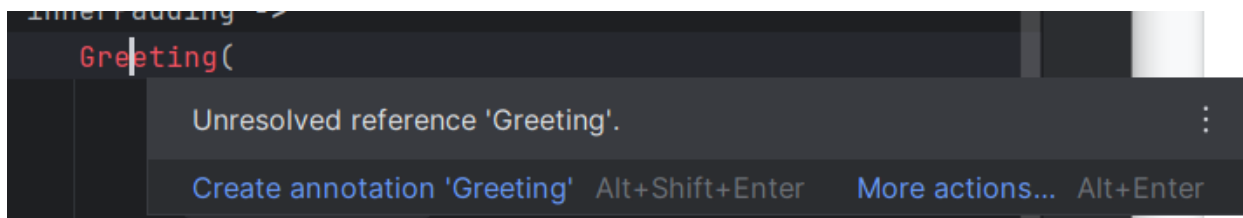
Шаг 8. Добавление текстового элемента

Удалите устаревшую функцию **Greeting()**:

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

После этого:

- Обратите внимание: в **onCreate()** и **RoyalInvitationPreview()** вызов **Greeting()** подсвечен красным — это ошибка.
- Наведите курсор на строку — **Android Studio** покажет подсказку.



Удалите вызовы функции **Greeting()** из **onCreate()** и **RoyalInvitationPreview()**:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PixLetterTheme {
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun RoyalInvitationPreview() {
    PixLetterTheme {
    }
}
```

Создание новой функции — **GreetingText()**

Ниже функции **RoyalInvitationPreview()** создайте новую компонентную функцию **GreetingText()**:

```
@Composable
fun GreetingText() {

}
```

Добавьте параметр **Modifier** в функцию **GreetingText()**:

```
@Composable
fun GreetingText(modifier: Modifier = Modifier) {

}
```

Добавьте параметр **message** типа **String** к компонуемой функции **GreetingText()**:

```
@Composable
fun GreetingText(message: String, modifier: Modifier = Modifier) {

}
```

В функции **GreetingText()** добавьте составную передачу текстового сообщения **Text** в качестве именованного аргумента:

```
@Composable
fun GreetingText(message: String, modifier: Modifier = Modifier) {
    Text(
        text = message
    )
}
```

Функция **отображает текст**, используя компонуемую функцию **Text()**

Добавим предварительный просмотр

Вызовите функцию **GreetingText()** внутри функции **RoyalInvitationPreview()**.

Передайте в качестве строкового аргумента функцию **GreetingText()** — приглашение от короля. Для переноса текста используйте спецсимвол **\n**. Сам текст:

Royal Decree!
Dear \$yourName,
His Majesty the King
Invites you to a great feast!
Come to the castle before sunset,
otherwise, you risk losing your head!


```

@Preview(showBackground = true)
@Composable
fun RoyalInvitationPreview() {
    PixLetterTheme {
        GreetingText(message = "Royal Decree!\n Dear Denis\nHis Majesty the
        King\nInvites you to a great feast!\nCome to the castle before
        sunset,\notherwise, you risk losing your head!")
    }
}

```

Предварительный просмотр должен выглядеть следующим образом:

```

Royal Decree!
Dear Denis
His Majesty the King
Invites you to a great feast!
Come to the castle before sunset,
otherwise you risk losing your head!

```

Шаг 9. Изменение размера шрифта

Масштабируемые пиксели

Масштабируемые пиксели (SP) — это единица измерения размера шрифта. Элементы пользовательского интерфейса в приложениях Android используют две разные единицы измерения: пиксели, не зависящие от плотности (DP) и масштабируемые пиксели (SP). По умолчанию единица SP имеет тот же размер, что и единица DP, но она изменяет размер в зависимости от предпочитаемого пользователем размера текста в настройках телефона.

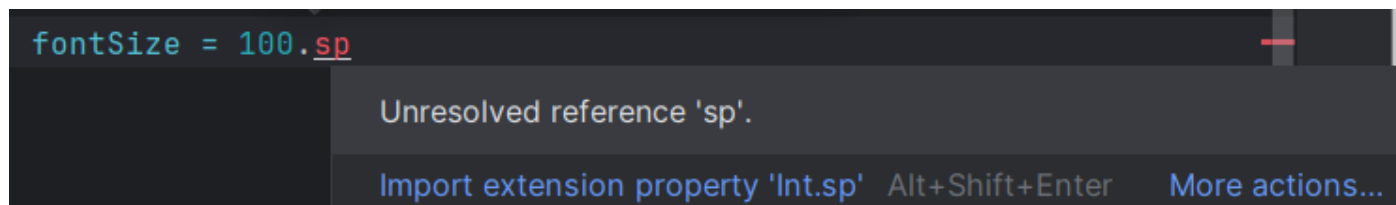
Внутри **GreetingText()** обновите вызов функции **Text()**, добавив второй именованный аргумент **fontSize** и присвойте ему значение **100.sp**:

```

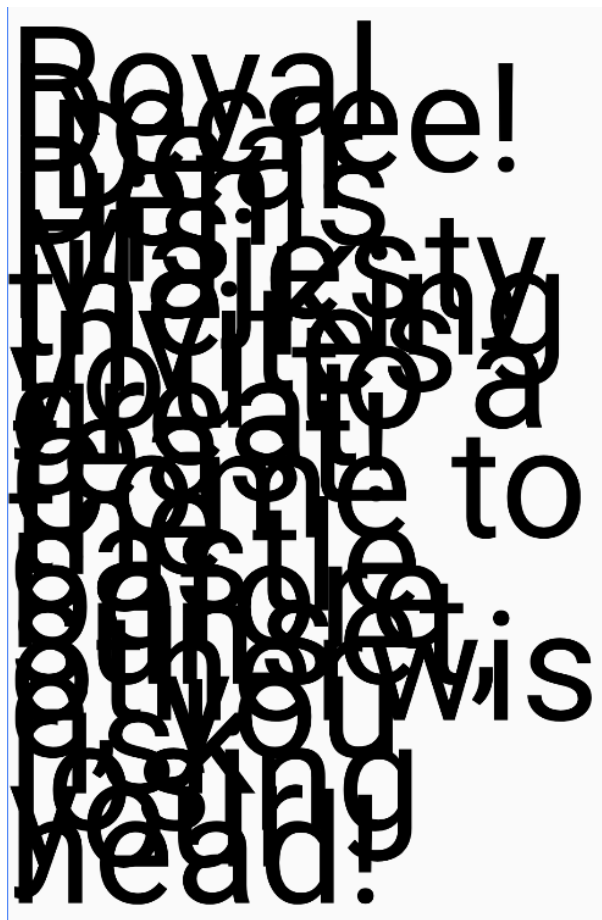
@Composable
fun GreetingText(message: String, modifier: Modifier = Modifier) {
    Text(
        text = message,
        fontSize = 100.sp
    )
}

```

При первом использовании **sp** **Android Studio** подсветит его красным. Нажмите **Alt+Enter** или воспользуйтесь всплывающим меню и выберите **Import** → **Импортируйте**:



Обратите внимание на обновленный предварительный просмотр:



Если текст **перекрывается**, укажите **lineHeight** — это высота строки. Добавьте его в **Text()**:

```
Text(  
    text = message,  
    fontSize = 100.sp,  
    lineHeight = 80.sp  
)
```

Это не устранил обрезание текста в **Preview**, но улучшит читаемость и внешний вид. Позже мы настроим **прокрутку** или **масштаб**, чтобы текст умещался:



Шаг 10. Добавляем текстовый элемент

Добавим **подпись от короля** в наше приглашение.

1. Добавьте новый параметр **from** типа **String** в функцию **GreetingText()**:

```
fun GreetingText(message: String, from: String, modifier: Modifier = Modifier) {  
    Text(  
        text = message,  
        fontSize = 100.sp,  
        lineHeight = 80.sp  
    )  
    Text(  
        text = from,  
        fontSize = 36.sp  
    )  
}
```

2. Внутри функции добавьте второй компонент **Text()** для отображения подписи:



3. Установите размер шрифта подписи, добавив аргумент **fontSize** со значением **36.sp**:

```
Text(
    text = from,
    fontSize = 36.sp
)
```

4. В функции **RoyalInvitationPreview()** передайте строку с именем короля:

From King Arthur

```
@Preview(showBackground = true)
@Composable
fun RoyalInvitationPreview() {
    PixLetterTheme {
        GreetingText(message = "Royal Decree!\n Dear Denis\nHis Majesty the
        King\nInvites you to a great feast!\nCome to the castle before
        sunset,\notherwise, you risk losing your head!", from = "From King
        Arthur")
    }
}
```

5. Обратите внимание на предварительный просмотр:



Compose позволяет описывать несколько UI-элементов внутри одной функции, но без указания правил их размещения элементы могут накладываться друг на друга (как в текущем примере с двумя `Text()`).

Расположение текстовых элементов в строке и столбце

Иерархия пользовательского интерфейса

В Compose используется древовидная структура, где:

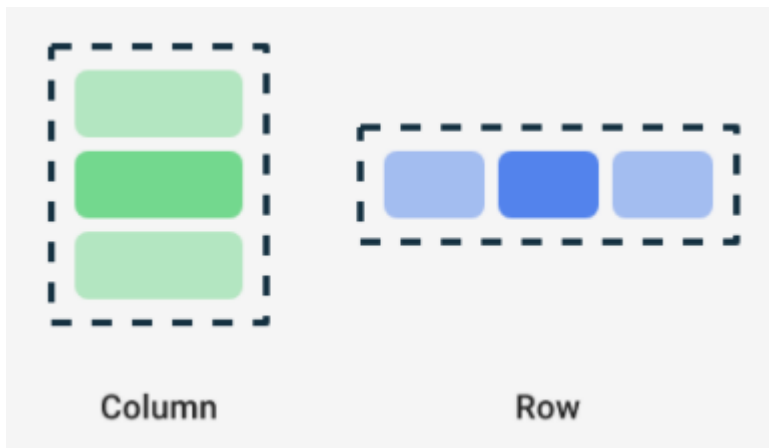
- **Родительский элемент** (например, контейнер) управляет расположением дочерних элементов.
- Дочерние элементы могут содержать другие элементы, образуя вложенную иерархию.



Основные контейнеры Compose

Для управления расположением элементов используются три стандартных компонента:

- **Column** — вертикальное расположение (столбец).
- **Row** — горизонтальное расположение (строка).
- **Box** — наложение элементов (слои).



Column, **Row**, и **Box** являются компонуемыми функциями, которые принимают компонуемое содержимое в качестве аргументов, поэтому вы можете размещать элементы внутри этих элементов макета. Например, каждый дочерний элемент внутри **Row** размещается горизонтально рядом друг с другом в строке:

```
Row {  
    Text("First Column")  
    Text("Second Column")  
}
```

Эти текстовые элементы отображаются на экране рядом друг с другом:



Синтаксис завершающей лямбды

В Kotlin есть упрощенный синтаксис для передачи функций как аргументов, если лямбда является последним параметром.



Когда вы передаете функцию в качестве этого параметра, вы можете использовать синтаксис завершающей лямбды. Вместо того, чтобы помещать функцию в скобки, вы можете поместить ее за скобки в фигурных скобках. Это рекомендуемая и распространенная практика в Compose.

Например, последний параметр в `Row()` компонентой функции — это `content` параметр, функция, описывающая дочерние элементы пользовательского интерфейса. Предположим, вы хотите создать строку, содержащую три текстовых элемента. Этот код будет работать, но очень неудобно использовать именованный параметр для завершающей лямбды:

```
Row(  
    content = {  
        Text("Some text")  
        Text("Some more text")  
        Text("Last text")  
    }  
)
```

Поскольку `content` параметр является последним в сигнатуре функции, а его значение передается как лямбда-выражение, вы можете удалить `content` параметр и скобки следующим образом:

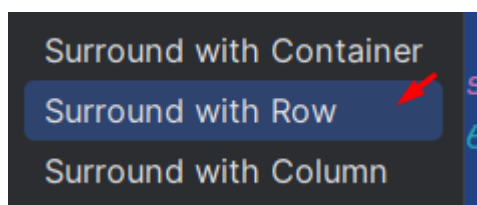
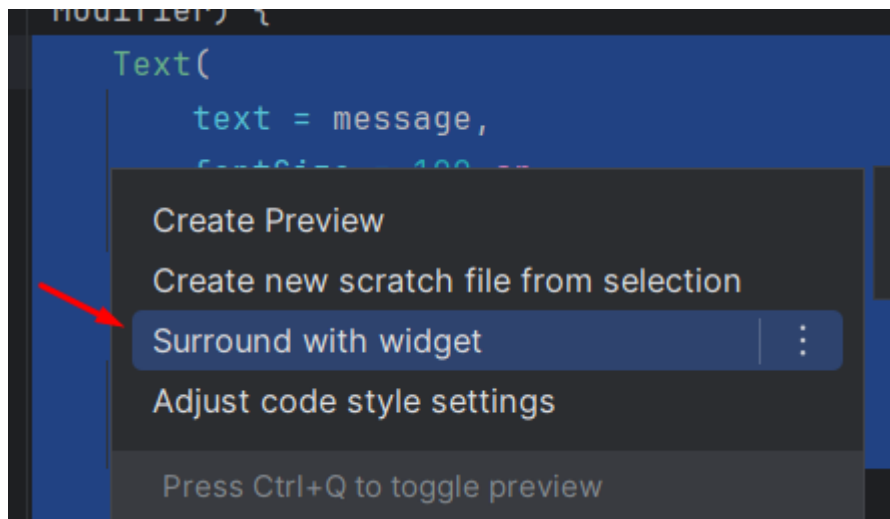
```
Row {  
    Text("Some text")  
    Text("Some more text")  
    Text("Last text")  
}
```

Шаг 11. Управление расположением элементов

Расположим текстовые элементы в своем приложении в ряд, чтобы избежать наложения.

В функции **`GreetingText()`** добавьте компонентый элемент **`Row`** вокруг текстовых элементов, чтобы он отображал строку с двумя текстовыми элементами. Для

быстро оборачивания выделяет **Text**, щелкаете на лампочку и выбираете **Surround with widget > Surround with Row**:



Теперь функция должна выглядеть как этот фрагмент кода:

```
@Composable
fun GreetingText(message: String, from: String, modifier:
    Modifier = Modifier) {
    Row {
        Text(
            text = message,
            fontSize = 100.sp,
            lineHeight = 80.sp
        )
        Text(
            text = from,
            fontSize = 36.sp
        )
    }
}
```

При этом у нас автоматически импортировался нужный пакет **androidx.compose.foundation.layout.Row**. В можете увидеть его в импорте:

```
7      import androidx.compose.foundation.layout.Row
8      import androidx.compose.foundation.layout.fillMaxSize
```

Не забудьте запустить **"Optimize imports"** (**Ctrl+Alt+O**), чтобы удалить неиспользуемые импорты и отсортировать существующие.

Поменяйте **размер шрифта на 20** и **высоту линии на 40**:

```
Row {  
  Text(  
    text = message,  
    fontSize = 20.sp,  
    lineHeight = 40.sp  
  )  
  Text(  
    text = from,
```

Обратите внимание как в предварительном просмотре у нас выглядит текст:

Royal Decree!	From King g Arthur
Dear Denis	
His Majesty the King	
Invites you to a great feast!	
Come to the castle before sunset,	
otherwise, you risk losing your head!	

Предварительный просмотр теперь **без перекрытий**, но подпись “**From King Arthur**” не читается должным образом, так как оба текста находятся в одной строке, и подписи недостаточно места.

В следующем шаге мы улучшим расположение — разместим элементы **один под другим**, используя **Column**, и добавим **отступы** для лучшего вида.

Шаг 12. Располагаем текстовые элементы в столбце

1. Замена Row на Column

Замените контейнер **Row** на **Column** для вертикального расположения текстовых элементов. Не забудьте импортировать пакет:

```
import androidx.compose.foundation.layout.Column
```

Готовый код должен выглядеть так:


```

@Composable
fun GreetingText(message: String, from: String, modifier:
    Modifier = Modifier) {
    Column {
        Text(
            text = message,
            fontSize = 20.sp,
            lineHeight = 40.sp
        )
        Text(
            text = from,
            fontSize = 36.sp
        )
    }
}

```

2. Передача модификатора

Модификатор должен передаваться от родительского компонента (**Column**) к дочерним элементам:

```

Modifier) {
    Column(modifier = modifier) {
        Text(

```



- Добавим в **onCreate()** текст сообщения и автора. Для этого обновите функцию **GreetingText()**:

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PixLetterTheme {
                GreetingText(message = "Royal Decree!\n Dear Denis\nHis Majesty the
                    King\nInvites you to a great feast!\nCome to the castle before
                    sunset,\notherwise, you risk losing your head!", from = "From King
                    Arthur")
            }
        }
    }
}

```

3. Центрирование содержимого

- Чтобы выровнять приветствие по центру экрана, добавьте параметр **verticalArrangement** выбрав **Arrangement.Center** в функцию **GreetingText()**:

```
@Composable
fun GreetingText(message: String, from: String, modifier: Modifier
= Modifier) {
    Column(
        verticalArrangement = Arrangement.Center,
        modifier = modifier
    ) {
```

- Добавьте **8.dp** отступ вокруг столбца:

```
Column(
    verticalArrangement = Arrangement.Center,
    modifier = modifier.padding(8.dp)) {
```

- Импортируйте отсутствующую библиотеку:

```
Arrangement.Center,
padding(8.dp)) {
```

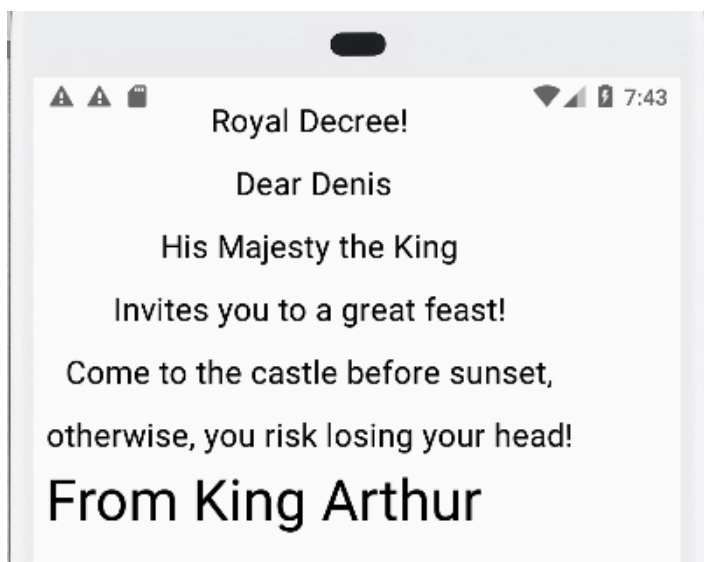
Unresolved reference 'dp'.

[Import extension property 'Int.dp'](#)

- Выровняйте текст приветствия по центру с помощью `textAlign`.

```
Text(
    text = message,
    fontSize = 20.sp,
    lineHeight = 40.sp,
    textAlign = TextAlign.Center
)
```

Итог:



На приведенном выше снимке экрана только приветствие выровнено по центру из-за `textAlign` параметра. Подпись **From King Arthur** имеет выравнивание по умолчанию слева.

Добавьте отступ к подписи и выровняйте ее по правому краю, можете также изменить размер шрифта:

```
        textAlign = TextAlign.Center
    )
    Text(
        text = from,
        fontSize = 24.sp,
        modifier = Modifier
            .padding(all = 16.dp)
            .align(alignment = Alignment.End)
    )
```

Использование передовой практики

Хорошей практикой является передача атрибута(ов) модификатора вместе с модификатором из родительского компонуемого объекта. Обновите параметр модификатора `GreetingText()` следующим образом:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            PixLetterTheme {
                GreetingText(
                    message = "Royal Decree!\n Dear Denis\nHis Majesty the King\nInvites you to a great feast!\nCome to the castle before sunset, \notherwise, you risk losing your head!",
                    from = "From King Arthur",
                    modifier = Modifier.padding(all = 8.dp)
                )
            }
        }
    }
}
```

Измените **GreetingText()**:

```
@Composable
fun GreetingText(message: String, from: String, modifier: Modifier
    = Modifier) {
    Column(
        verticalArrangement = Arrangement.Center,
        modifier = modifier
    ) {
```

Соберите и запустите свое приложение на эмуляторе, чтобы увидеть конечный результат.

